Parameter	Informed Search	Uninformed Search
Utilizing Knowledge	Uses knowledge (e.g., heuristic) during searching	Does not use any additional knowledge
Speed	Faster	Slower
Completeness	May or may not be complete	Always complete (if finite space)
Time Consumption	Less time	More time
Cost	Lower cost	Higher cost
Guidance	Gets direction on where to search	No guidance; relies only on problem definition
Efficiency	More efficient	Less efficient
Implementation	Shorter implementation	Longer implementation
Examples	A*, Greedy Best-First Search	BFS, DFS, Uniform-Cost Search

2. Define heuristic function? Give two examples.

- A **heuristic function**, denoted as $h(n)$, estimates the cost of the cheapest path from the state at node n to a goal state.
- It helps in making the search process more efficient by guiding the search toward the goal.
- **Two examples:**
  1. **Straight-line distance** in route-finding problems (e.g., Romania map).
  2. **Number of misplaced tiles** in the 8-puzzle problem.

3. How do you evaluate the performance of a search algorithm?

Performance is evaluated using:

- **Completeness:** Does it always find a solution if one exists?
- **Optimality:** Does it find the best solution (lowest path cost)?
- **Time Complexity:** How long does it take to find a solution?
- **Space Complexity:** How much memory is required?
- **Effective Branching Factor:** Estimated branching factor based on nodes expanded.

---

4. Describe the state space representation of 8-queens problem.

- **States:** Any arrangement of 0 to 8 queens on the chessboard.
- **Initial State:** No queens on the board.
- **Actions:** Add a queen to any empty square.
- **Transition Model:** Returns the board with a queen added to the specified square.
- **Goal Test:** 8 queens are placed such that none attack each other.

---

5. Discuss the infrastructure needed to solve a search problem.

The infrastructure includes:

- **Initial State:** The starting point of the agent.
- **Actions:** Possible moves or operations.
- **Transition Model:** Describes the result of applying an action to a state.
- **Goal Test:** Determines if a state is the goal.
- **Path Cost Function:** Assigns a cost to each path.
- **Search Strategy:** Algorithm to choose the order of node expansion.

6. Explain the concept of Uniform Cost Search.

- Uniform Cost Search (UCS) expands the node with the **lowest path cost** $g(n)$.
- It uses a **priority queue** ordered by $g(n)$.
- **Optimal** and **complete** if step costs are non-negative.
- Similar to BFS when all step costs are equal, but continues to check all nodes at the goal depth for optimality.

---

7. List any three advantages of Depth First Search.

- Requires **less memory** than BFS (only stores path from root to current node).
- May find a solution **without examining the entire state space**.
- **Simple to implement** (using recursion or stack).

---

8. Describe the difference between admissible heuristic and consistent with the help of an example.

Admissible Heuristic	Consistent Heuristic
Never overestimates the true cost to the goal: $h(n) \leq h^*(n)$	Satisfies the triangle inequality: $h(n) \leq c(n, a, n') + h(n')$
Required for **tree search** optimality in A*	Required for **graph search** optimality in A*
Example: **Straight-line distance** in routing (never overestimates actual road distance)	Example: Same heuristic, but also satisfies consistency condition for all nodes

9. List the differences between tree search algorithm and graph search algorithm.

Tree Search	Graph Search
May revisit states (can get stuck in loops)	Uses a **closed list** to avoid revisiting states
Can be inefficient in cyclic state spaces	More efficient in cyclic spaces
No memory of visited nodes	Maintains a record of explored nodes
Example: Basic DFS, BFS without tracking visited states	Example: A* with closed list to prevent duplicates

1. Explain Best First Search algorithm. How does it implement heuristic search? (6 Marks)

- **Concept:** Best-First Search (BFS) is an informed search algorithm that explores a graph by expanding the most promising node chosen according to a specified rule.
- **Heuristic Implementation:** It uses an **evaluation function**, denoted as $f(n)$, to estimate the promise of a node. The node with the most promising (lowest) value of $f(n)$ is selected for expansion first.
- **Priority Queue:** It is implemented using a **priority queue** (often called the OPEN list) where nodes are ordered by their $f(n)$ value.
- **Core Idea:** It combines the systematic nature of Breadth-First Search with the guidance of a heuristic, allowing it to "jump" towards the goal rather than exploring all paths blindly.
- **Greedy Best-First Search:** A common variant is Greedy Best-First Search, where the evaluation function is simply the heuristic function itself: $f(n) = h(n)$. This makes it choose the node that appears to be closest to the goal.
- **Process:**
  1. Start with the initial node in the OPEN list.
  2. While the OPEN list is not empty:
     a. Remove the node `n` with the best (lowest) $f(n)$ from OPEN and put it in the CLOSED list.
     b. If `n` is the goal, return the solution.
     c. Else, expand `n` to generate all its successors.
     d. For each successor, calculate its $f(n)$ and add it to the OPEN list if it hasn't been visited.

2. Write and explain A* search algorithm (6 Marks)

- **Objective:** A* search aims to find the shortest path from a start node to a goal node. It is both **complete** and **optimal** under certain conditions.
- **Evaluation Function:** It uses a sophisticated evaluation function: $f(n) = g(n) + h(n)$, where:
  - $g(n)$: The actual cost from the start node to node `n`.
  - $h(n)$: The estimated cost (heuristic) from node `n` to the goal.
- **Algorithm Steps:**

  1. **Initialize:** Place the starting node in the OPEN list with its $f(n) = g(n) + h(n)$.
  2. **Loop:**

     a. If OPEN is empty, return failure.

     b. Select and remove the node `n` from OPEN with the lowest $f(n)$. Place it in the CLOSED list.

     c. If `n` is the goal, trace back the path to the start and return success.

     d. Else, expand `n` to generate all its successors.

     e. For each successor `n'`:

     * Calculate $g(n')$, $h(n')$, and $f(n')$.

     * If `n'` is not in OPEN or CLOSED, add it to OPEN.

     * If `n'` is already in OPEN/CLOSED with a higher $g(n')$, update its cost to the lower value.
  3. **Terminate:** Repeat until the goal is found or OPEN is empty.

3. State the different conditions for optimality for A* search. Explain A* search with an example (6 Marks)

- **Conditions for Optimality:**
  1. **Admissibility:** The heuristic function $h(n)$ must be **admissible**, meaning it never overestimates the true cost to reach the goal. Formally, $h(n) \leq h^*(n)$, where $h^*(n)$ is the true cost.
  2. **Consistency (or Monotonicity):** The heuristic must be **consistent**. For every node `n` and its successor `n'` generated by action `a`, the estimated cost of reaching the goal from `n` should be no greater than the step cost of getting to `n'` plus the estimated cost from `n'` to the goal. Formally, $h(n) \leq c(n, a, n') + h(n')$. Consistency implies admissibility.
- **Example (Romania Route Finding):**
  - **Goal:** Reach Bucharest from Arad.
  - **Heuristic $h(n)$:** Straight-line distance to Bucharest (admissible and consistent).
  - **Step Cost $g(n)$:** Actual road distance traveled.
  - **Process:**
    1. Start at Arad: $f = 0 + 366 = 366$.
    2. Expand Arad. Successors: Sibiu ($f = 140 + 253 = 393$), Timisoara (447), Zerind (449). Choose Sibiu (lowest f).
    3. Expand Sibiu. Successors include Rimnicu Vilcea ($f = 220 + 193 = 413$) and Fagaras ($f = 239 + 176 = 415$). Choose Rimnicu Vilcea.
    4. Continue expanding the node with the lowest f. The algorithm will find the path Arad → Sibiu → Rimnicu Vilcea → Pitesti → Bucharest with a cost of 418, which is the optimal path.

4. Explain and compare the Breadth first search and Uniform cost search with the uninformed search evaluation strategies such as completeness, optimality, space and time complexities. (6 Marks)

Criterion	Breadth-First Search (BFS)	Uniform-Cost Search (UCS)
Strategy	Expands shallowest nodes first (FIFO queue).	Expands node with lowest path cost $g(n)$ (priority queue).
Completeness	Yes, if branching factor `b` is finite.	Yes, if step costs are ≥ ε (a small positive value).
Optimality	Yes, **only if all step costs are identical.**	Yes, always optimal for any step cost.
Time Complexity	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$, where C^* is the optimal cost.
Space Complexity	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$
Key Difference	Optimal only for uniform cost. Stops at goal generation.	Always optimal. Checks all nodes at goal depth for a cheaper path.

4. Explain and compare the Breadth first search and Uniform cost search with the uninformed search evaluation strategies such as completeness, optimality, space and time complexities. (6 Marks)

Criterion	Breadth-First Search (BFS)	Uniform-Cost Search (UCS)
Strategy	Expands shallowest nodes first (FIFO queue).	Expands node with lowest path cost $g(n)$ (priority queue).
Completeness	Yes, if branching factor `b` is finite.	Yes, if step costs are ≥ ε (a small positive value).
Optimality	Yes, **only if all step costs are identical.**	Yes, always optimal for any step cost.
Time Complexity	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$, where C^* is the optimal cost.
Space Complexity	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$
Key Difference	Optimal only for uniform cost. Stops at goal generation.	Always optimal. Checks all nodes at goal depth for a cheaper path.

5. Discuss the optimality of A* algorithm (6 Marks)

- **Foundation:** A* is **optimal** if its heuristic function $h(n)$ is **admissible** (for tree search) and **consistent** (for graph search, to avoid re-expanding nodes).
- **Mechanism:** The evaluation function $f(n) = g(n) + h(n)$ provides an optimistic estimate of the total cost of a path going through node `n`. Because $h(n)$ is admissible, $f(n)$ never overestimates the true cost of the best path through `n`.
- **Optimal Efficiency:** A* is **optimally efficient** for any given consistent heuristic. This means no other optimal algorithm is guaranteed to expand fewer nodes than A*. This is because any algorithm that does not expand all nodes with $f(n) < C^*$ (the optimal solution cost) risks missing the optimal solution.
- **Proof Sketch:** A* expands nodes in **non-decreasing order of f-cost**. Therefore, when it selects a goal node for expansion, its `f` value (which equals its `g` value, since $h(goal) = 0$) is the lowest possible, meaning it has found the optimal path.

6. Discuss any two uninformed search strategies (6 Marks)

1. Depth-First Search (DFS):

- **Strategy:** Expands the deepest node in the current frontier first. Uses a **LIFO stack** (or recursion).
- **Completeness:** No (can get stuck in infinite loops in infinite state spaces). Yes, in finite state spaces.
- **Optimality:** No.
- **Time Complexity:** $O(b^m)$, where `m` is the maximum depth (can be much larger than `d`).
- **Space Complexity:** $O(bm)$ (only stores a single path, very memory efficient).
- **Advantage:** Low memory requirement.

2. Iterative Deepening Search (IDS):

- **Strategy:** Combines the benefits of BFS and DFS. It performs a series of DFS searches with increasing depth limits (0, 1, 2,...) until the goal is found.
- **Completeness:** Yes.
- **Optimality:** Yes, if step costs are identical.
- **Time Complexity:** $O(b^d)$ (similar to BFS).
- **Space Complexity:** $O(bd)$ (similar to DFS).
- **Advantage:** Optimal and complete like BFS, but with the low memory footprint of DFS. Avoids the infinite loop problem of DFS.

7. Discuss the iterative deepening search with an example. (6 Marks)

- **Concept:** IDS is an uninformed search strategy that runs a series of Depth-Limited Searches (DLS) with increasing depth limits. It starts with a limit of 0, then 1, then 2, and so on, until the goal is found.
- **Purpose:** It overcomes the potential infinite path problem of DFS while maintaining DFS's low space complexity.
- **Process:** For each iteration, it performs a DFS but does not expand nodes beyond the current depth limit `l`. If the goal is not found, `l` is increased and the search starts over.
- **Example:**
  Consider a tree with root A, and goal node E at depth 3.
  - **Depth Limit = 0:** Explore only A. Goal not found.
  - **Depth Limit = 1:** Explore A → B → C. (Then backtrack). Goal not found.
  - **Depth Limit = 2:** Explore A → B → D → E → ... (But at depth 2, E's children won't be generated). Wait, E is at depth 3, so it won't be found here. The search will backtrack and explore other branches up to depth 2.
  - **Depth Limit = 3:** Explore A → B → D → E. Goal found.

Although nodes near the root (like B) are expanded multiple times, the overhead is not large compared to the total number of nodes, and the algorithm successfully finds the goal with minimal memory.

8. Discuss the heuristic function. Explain how the heuristic function helps during search procedure. Explain with a suitable example (6 Marks)

- **Definition:** A heuristic function, $h(n)$, is a function that **estimates** the cost of the cheapest path from a given node `n` to the nearest goal state. It uses problem-specific knowledge.
- **Role in Search:** It guides the search process by providing a "hint" or "best guess" about which path is most likely to lead to the goal quickly and cheaply. It helps to **prune** unpromising paths, drastically reducing the number of nodes that need to be expanded.
- **Example - 8-Puzzle:**
  - **Problem:** Rearrange tiles from a start state to a goal state.
  - **Heuristic 1 (h1): Number of Misplaced Tiles.** Counts how many tiles are not in their goal position. It is admissible.
  - **Heuristic 2 (h2): Manhattan Distance.** Sums the horizontal and vertical distances each tile is from its goal position. This is more informed than h1 and is also admissible.
  - **How it helps:** An algorithm like A* using Manhattan distance will prioritize moving a tile that is far from its goal, leading to a much more efficient search compared to an uninformed search which might waste time moving tiles that are already correct.

9. Evaluate a problem as a state space search with an example (6 Marks)

- **State Space Representation:** Formulating a problem for AI involves defining:
  1. **Initial State:** The starting configuration.
  2. **Actions:** The set of all possible moves/operations from a state.
  3. **Transition Model:** The result of applying an action to a state.
  4. **Goal Test:** Checks if a given state is a goal state.
  5. **Path Cost:** A function that assigns a cost to each path (sequence of actions).
- **Example: 8-Puzzle Problem**
  - **States:** Every possible arrangement of the 8 tiles and the blank in the 3x3 grid.
  - **Initial State:** Any given configuration (e.g., tiles 7, 2, 4; 5, blank, 6; 8, 3, 1).
  - **Actions:** Move the blank space *Left, Right, Up, Down* (where possible).
  - **Transition Model:** Defines the new state after a legal move (e.g., moving blank left swaps it with the tile to its left).
  - **Goal Test:** The state matches the goal configuration (e.g., tiles 1-8 in order).
  - **Path Cost:** Each move costs 1, so the total cost is the number of moves.

10. List and explain the parameters for evaluating the performance of an AI search algorithm. (6 Marks)

1. **Completeness:** Is the algorithm guaranteed to find a solution if one exists? (Yes/No).
2. **Optimality:** Does the algorithm find the highest-quality solution (typically the one with the lowest path cost)? (Yes/No).
3. **Time Complexity:** How long does it take to find a solution? Usually measured by the number of nodes generated/expanded. Expressed in Big-O notation (e.g., $O(b^d)$).
4. **Space Complexity:** How much memory is required to perform the search? Measured by the maximum number of nodes stored in memory simultaneously. Also expressed in Big-O notation.
5. **Effective Branching Factor (b*):** A more refined measure of time/space complexity. It is the average number of successors of a node in the problem space. If an algorithm expands N nodes to find a solution at depth d, then $N = 1 + b^* + (b^*)^2 + ... + (b^*)^d$. A lower b* indicates a more efficient heuristic.

11. Illustrate how a heuristic can be derived from a relaxed problem, using the example of 8 puzzle problem. (6 Marks)

- **Concept:** A powerful way to create admissible heuristics is to define a **relaxed problem**—a simpler version of the original problem where some rules are removed. The cost of an optimal solution to the relaxed problem is often a good heuristic for the original problem.
- **8-Puzzle Relaxations:**
  1. **Relaxation A:** A tile can move from square A to square B if A is adjacent to B. (We remove the constraint that B must be blank).
     - **Heuristic Derived (h^2):** The solution to this relaxed problem is the **Manhattan distance.** A tile can simply "hop" to its goal square, and the number of moves is the sum of the horizontal and vertical distances for all tiles.
  2. **Relaxation B:** A tile can move from square A to square B if B is the blank space. (We remove the adjacency constraint).
     - **Heuristic Derived:** This is not very useful, as any tile can swap with the blank anywhere.
  3. **Relaxation C:** A tile can move from any square A to any square B.
     - **Heuristic Derived (h^1):** The solution to this is the **number of misplaced tiles**, as each misplaced tile can move directly to its goal in one step.

Since the relaxed problems have fewer restrictions, their solution costs are guaranteed to be less than or equal to the cost of the original problem, ensuring the derived heuristics are **admissible.**

12. Explain the algorithm for depth limited search with an example. (6 Marks)

- **Concept:** Depth-Limited Search (DLS) is a variation of DFS that imposes a cutoff depth `l` on the search. Nodes at depth `l` are treated as if they have no successors, preventing the search from going down infinitely deep paths.
- **Algorithm (Recursive):**

python Copy Download

```python
function DLS(node, problem, limit):
    if problem.GOAL-TEST(node.STATE):
        return SOLUTION(node)
    else if limit == 0:
        return 'cutoff'
    else:
        cutoff_occurred = False
        for each action in problem.ACTIONS(node.STATE):
            child = CHILD-NODE(problem, node, action)
            result = DLS(child, problem, limit - 1)
            if result == 'cutoff':
                cutoff_occurred = True
            else if result != failure:
                return result
        if cutoff_occurred:
            return 'cutoff'
        else:
            return 'failure'
```

- **Example:**
  Consider a tree with a depth limit `l = 2`. The root is at level 0.
  1. Expand Root (Level 0).
  2. Expand its children (Level 1).
  3. Expand the children of Level 1 nodes (Level 2).
  4. Any nodes at Level 2 will not be expanded because the limit is reached. The algorithm will return 'cutoff' for those branches.
     If the goal was at Level 3, DLS with `l=2` would fail to find it, demonstrating its **incompleteness** if `l < d` (the goal depth).

1. Describe any four uninformed search strategies. (8 Marks)

1. Breadth-First Search (BFS)

- **Strategy:** Expands the root node first, then all of its successors, then *their* successors, and so on. It explores the tree level by level.
- **Implementation:** Uses a **FIFO (First-In-First-Out) queue** for the frontier. New nodes are added to the end of the queue.
- **Properties:**
  - **Completeness:** Yes, if the branching factor `b` is finite.
  - **Optimality:** Yes, but only if all step costs are identical.
  - **Time Complexity:** $O(b^d)$
  - **Space Complexity:** $O(b^d)$ (Major drawback, as it consumes a lot of memory).

2. Depth-First Search (DFS)

- **Strategy:** Always expands the **deepest node** in the current frontier of the search tree. It goes as deep as possible down one path before backtracking.
- **Implementation:** Uses a **LIFO (Last-In-First-Out) stack** (or recursion) for the frontier. The most recently generated node is expanded first.
- **Properties:**
  - **Completeness:** No in infinite state spaces (can get stuck in infinite loops). Yes in finite state spaces.
  - **Optimality:** No.
  - **Time Complexity:** $O(b^m)$, where `m` is the maximum depth (can be much larger than `d`).
  - **Space Complexity:** $O(bm)$ (Major advantage, requires much less memory than BFS).

3. Depth-Limited Search (DLS)

3. Depth-Limited Search (DLS)

- **Strategy:** A variation of DFS that imposes a predetermined depth limit `l`. Nodes at depth `l` are treated as if they have no successors, preventing infinite loops.
- **Implementation:** Similar to DFS, but the search terminates and backtracks when the depth limit is reached.
- **Properties:**
  - **Completeness:** No, if the shallowest goal is deeper than the depth limit `l`.
  - **Optimality:** No.
  - **Time Complexity:** $O(b^l)$
  - **Space Complexity:** $O(bl)$

4. Iterative Deepening Depth-First Search (IDS)

- **Strategy:** Combines the benefits of BFS and DFS. It performs a series of DLS searches with increasing depth limits (0, 1, 2, ...) until the goal is found.
- **Implementation:** Repeatedly calls DLS, incrementing the depth limit each time.
- **Properties:**
  - **Completeness:** Yes.
  - **Optimality:** Yes, if step costs are identical.
  - **Time Complexity:** $O(b^d)$ (similar to BFS).
  - **Space Complexity:** $O(bd)$ (similar to DFS). It is the preferred uninformed search method when the search space is large and the solution depth is unknown.

2. Explain the components of a well-defined AI problem? Write the standard formulation of 8-puzzle problem. (8 Marks)

A well-defined problem can be formally defined by **five components**:

1. **Initial State:** The state in which the agent starts. (e.g., `In(Arad)`).
2. **Actions:** The set of possible actions available to the agent. Given a state `s`, `ACTIONS(s)` returns the actions applicable in `s`. (e.g., `{Go(Sibiu), Go(Timisoara), Go(Zerind)}`).
3. **Transition Model:** A description of what each action does. Specified by a `RESULT(s, a)` function that returns the state that results from doing action `a` in state `s`. (e.g., `RESULT(In(Arad), Go(Zerind)) = In(Zerind)`). A state reachable by a single action is called a **successor**.
4. **Goal Test:** Determines whether a given state is a goal state. It can be an explicit set of states or a property to be satisfied. (e.g., `In(Bucharest)`).
5. **Path Cost Function:** A function that assigns a numeric cost to each path (sequence of actions). It is often the sum of the costs of the individual actions (step costs). An **optimal solution** has the lowest path cost among all solutions.

Standard Formulation of the 8-Puzzle Problem:

- **States:** Every possible arrangement of the eight numbered tiles and the blank space on the 3x3 board.
- **Initial State:** Any state can be designated as the initial state.
- **Actions:** The movements of the blank space: *Left, Right, Up, Down*. The subset of these actions that are possible depends on the current position of the blank.
- **Transition Model:** Given a state and action, returns the resulting state. For example, applying *Left* to a state moves the blank left, swapping it with the tile to its left.
- **Goal Test:** Checks if the state matches the canonical goal configuration (tiles 1-8 in order, with the blank in the center or a corner).
- **Path Cost:** Each move (slide) costs 1. The path cost is the total number of moves.

3. Explain how the problem of 8-puzzle can be solved with the help of heuristics. (8 Marks)

Heuristics are used in **informed search** algorithms (like A* or Greedy Best-First Search) to solve the 8-puzzle more efficiently by guiding the search towards the goal.

- **Concept:** A heuristic function, $h(n)$, estimates the cost from a state `n` to the goal. For the 8-puzzle, good heuristics should be **admissible** (never overestimate the true cost).
- **Common Heuristics for 8-Puzzle:**
  1. **h1: Number of Misplaced Tiles:** This heuristic counts how many tiles are not in their goal position.
     - **Example:** If 5 tiles are out of place, $h1(n) = 5$.
     - **Admissibility:** It is admissible because any misplaced tile must be moved at least once.
  2. **h2: Manhattan Distance:** This heuristic sums the horizontal and vertical distances each tile is from its goal position.
     - **Example:** A tile in the top-right corner that belongs in the center has a Manhattan distance of 2 (1 right + 1 down).
     - **Calculation:** For all 8 tiles, $h2(n) = \sum_{i=1}^{8} |x_i - x_{goal}| + |y_i - y_{goal}|$.
     - **Admissibility:** It is admissible because each tile must be moved at least its Manhattan distance to reach its goal.
- **How it Helps in Search:**
  An algorithm like *A search** uses the evaluation function $f(n) = g(n) + h(n)$, where `g(n)` is the cost to reach node `n`.
  - Without a heuristic (uninformed search), the algorithm might explore many irrelevant states.
  - With a heuristic like Manhattan distance, the algorithm prioritizes states where the tiles are, on average, closer to their goal positions. This **prunes** large parts of the search tree, leading to a dramatic reduction in the number of nodes expanded and allowing the solution to be found much faster.

4. Explain the working of Best First Search with an example. (8 Marks)

- **Concept:** Best-First Search is an informed search algorithm that expands the most promising node based on an evaluation function $f(n)$. A common variant is **Greedy Best-First Search**, where $f(n) = h(n)$.
- **Algorithm Steps:**
  1. Place the starting node in the OPEN list.
  2. If the OPEN list is empty, return failure.
  3. Remove the node `n` from OPEN with the lowest $h(n)$ and place it in the CLOSED list.
  4. If `n` is the goal node, return success and trace the path.
  5. Else, expand `n` to generate all its successors.
  6. For each successor, if it is not in OPEN or CLOSED, add it to the OPEN list.
  7. Return to Step 2.
- **Example (from PDF):**
  Let the heuristic values be: S=13, A=12, B=4, C=7, D=3, E=8, F=2, H=4, I=9, G=0.
  - **Initialization:** OPEN = [S], CLOSED = [].
  - **Iteration 1:** Expand S. OPEN = [A(12), B(4)], CLOSED = [S].
  - **Iteration 2:** Expand B (lowest h=4). OPEN = [A(12), E(8), F(2)], CLOSED = [S, B].
  - **Iteration 3:** Expand F (lowest h=2). OPEN = [A(12), E(8), H(4), I(9), G(0)], CLOSED = [S, B, F].
  - **Iteration 4:** Expand G (h=0). It is the goal. Solution found.
  - **Solution Path:** S → B → F → G.

Note: While this found a solution quickly, Greedy Best-First Search is **not optimal** and **not complete** (can get stuck in loops).

5. How is Depth Limited Search different from Depth First Search? Explain the steps followed in both cases, using an example. (8 Marks)

Feature	Depth-First Search (DFS)	Depth-Limited Search (DLS)
Depth Control	No depth limit. Can follow a path infinitely deep.	Has a predefined depth limit `l`. Stops at that depth.
Completeness	Not complete in infinite spaces (can loop).	Not complete if goal depth `d > l`.
Infinite Loop	Can get stuck in an infinite path.	Solves the infinite path problem of DFS.
Steps	1. Start at root. 2. Go to first child. 3. Repeat until a dead-end. 4. Backtrack to last unexplored node.	1. Start at root with limit `l`. 2. Go to first child, decrementing depth. 3. Repeat until depth `l` is reached. 4. Treat nodes at depth `l` as dead-ends and backtrack.

Example:
Consider a tree where the goal is at depth 3, but there is an infinite branch at depth 1.

- **DFS Behavior:** It might start down the infinite branch and **never reach the goal.**
- **DLS Behavior (with limit l=3):** It will ignore paths beyond depth 3. It will explore the infinite branch only up to depth 3, then backtrack, and eventually find the goal at depth 3. If the limit was set to `l=2`, DLS would also fail to find the goal.

6. Discuss the difference between Tree Search and Graph search strategies for finding solutions to AI problems. Write the pseudocodes for Tree Search and Graph Search. (8 Marks)

The key difference lies in how they handle **repeated states**.

Aspect	Tree Search	Graph Search
State Tracking	Does not remember visited states. It can expand the same state multiple times if it is reached via different paths.	Maintains a **closed list** (or explored set) to keep track of all visited states. It never expands a state twice.
Completeness	May not be complete in finite state spaces if it gets stuck in a cycle.	Complete in finite state spaces, as it avoids cycles.
Optimality	Can be non-optimal if a shorter path to a node is found after it has already been expanded.	Can be optimal (e.g., with Uniform-Cost Search or A* with a consistent heuristic).
Space Complexity	Lower, as it doesn't store the closed list.	Higher, due to the memory required for the closed list.
Efficiency	Can be very inefficient due to redundant work.	More efficient in state spaces with many cycles, as it avoids re-expanding states.

Pseudocode

Pseudocode

Tree-Search:

text Copy Download

```text
function TREE-SEARCH(problem) returns a solution, or failure
    frontier <- a data structure containing the initial state of the problem
    loop do
        if frontier is empty then return failure
        choose a leaf node and remove it from the frontier
        if the node contains a goal state then return the corresponding solution
        expand the chosen node, adding the resulting nodes to the frontier
```

Graph-Search:

text Copy Download

```text
function GRAPH-SEARCH(problem) returns a solution, or failure
    frontier <- a data structure containing the initial state of the problem
    explored <- an empty set
    loop do
        if frontier is empty then return failure
        choose a leaf node and remove it from the frontier
        if the node contains a goal state then return the corresponding solution
        add the node to the explored set
        expand the chosen node, adding the resulting nodes to the frontier
           only if not in the frontier or explored set
```